

# dev-workflow 使用分享

这篇分享面向 dev-workflow 作者和实践者。重点不是介绍“怎么安装一个工具”，而是讨论：我如何理解 dev-workflow 的设计、在真实项目里怎么拆解和使用它、过程中遇到了哪些问题，以及哪些问题值得继续和大家一起讨论。

## Harness Engineering/AI Workflow Governance

关于 Harness Engineering 的基本判断，大家大概率已经有共识：AI Coding 的问题不只是“模型会不会写代码”，而是能不能在真实项目里按规则、按流程、可验证地交付。

所以这篇分享不再从“为什么需要 Harness”展开，而是直接从我对 dev-workflow 的拆解开始：它到底把研发流程拆成了哪些层，每一层解决什么问题，实践中哪些设计让我觉得有价值，哪些地方还值得继续讨论。

我的一句话结论是：dev-workflow 的价值不在于多写几行代码，而在于把需求分析、知识注入、技术方案、代码生成、测试验证、归档复盘这些工程动作，组织成 AI 可以执行、人可以检查、失败可以恢复的流程。

### 1. 我对 dev-workflow 的整体拆解

我把当前 dev-workflow 拆成六层来看。

#### 1.1 命令层：给用户稳定入口

仓库中的 `commands/` 把完整研发链路收敛成几个入口：

| 代码块 |                        |                   |
|-----|------------------------|-------------------|
| 1   | 00-init                | 环境初始化             |
| 2   | 01-knowledge-base      | 构建应用知识库           |
| 3   | 02-implementation-plan | MRD -> PRD + 技术方案 |
| 4   | 03-code-gen-tdd        | 代码生成 + TDD 验证闭环   |
| 5   | 04-archive             | 归档 + 知识库更新        |

这层解决的是“用户怎么触发”的问题。使用者不需要理解所有 Agent，只需要知道当前要做的是初始化、知识库、方案、编码还是归档。

#### 1.2 Skill 层：定义流程契约

`skills/` 是我认为最关键的一层。它不是简单提示词，而是流程契约。

例如 `03-code-gen-tdd` 明确规定：

- Orchestrator 只负责阶段编排、确认门、状态写入。
- 正式产物必须由专职 Agent 生成。
- 每个 Phase 必须先输出执行契约，再调度唯一允许的 Agent。
- Task prompt 只传路径和小体积元数据，禁止内联大文本。
- 主会话禁止直接生成代码、测试、Review 报告或测试报告。

这其实是在约束 AI 的越权行为。没有这层约束，AI 很容易为了图快直接改代码、直接跑测试、直接补测试，最后看似完成，实际产物来源、验证链路和责任边界都不可追溯。

### 1.3 Agent 层：把角色拆成单一职责

`agents/` 更像一个研发团队的岗位拆分：

| Agent           | 职责           |
|-----------------|--------------|
| app-knowledge   | 扫描应用，生成知识库   |
| mrd-clarify     | 澄清 MRD 和不确定点 |
| prd-generator   | 生成产品可读 PRD   |
| tech-design     | 生成技术方案       |
| java-impl       | 实现生产代码       |
| java-review     | 做增量代码审查      |
| testcode-gen    | 生成测试代码       |
| tdd-test-runner | 执行测试和覆盖率诊断   |
| kb-update       | 归档后更新知识库     |
| archive-report  | 生成需求跟踪报告     |

这种拆分带来的收益是上下文更干净、职责更清楚、失败更容易定位。代价是流程变重了，状态维护和调度复杂度也明显增加。

### 1.4 Rules 层：把团队经验显性化

`rules/` 里沉淀的是团队工程约束，例如 Java 编码规范、测试规范、Git 工作流、上下文管理、多 Agent 协作规则等。

这层非常重要。AI 如果只靠通用经验写代码，很难贴合具体团队的工程习惯。Rules 的价值在于把隐性经验变成可读取、可复用、可 Review 的工程资产。

比如在实践中，我们会自然希望这些内容进入规则层：

- 缓存应该怎么用，哪些场景必须设置 TTL。
- 注释应该写到什么程度，哪些代码必须说明业务原因。
- 金额、时间、枚举、状态机等字段的约束。
- 外部调用是否必须超时、降级、记录关键日志。
- 代码风格如何贴近当前项目，而不是写成模型自己的风格。

## 1.5 State 层：让长流程可以恢复

`execution-state.md` 是 dev-workflow 中很关键的设计。它记录：

- 当前完成到哪个阶段。
- 下一个阶段是什么。
- 关键产物路径。
- Phase 门禁状态。
- 测试和覆盖率重试次数。
- 用户偏好和持久记忆。

这解决了 AI 长流程里很现实的问题：上下文会膨胀，会中断，会失败，会需要续跑。把状态落盘后，流程不再完全依赖对话历史，而是可以从文件恢复。

## 1.6 增强工具层：把 AI 放进工程系统

dev-workflow 还接入了 OpenSpec、Beads、GitNexus、autoresearch、飞书 MCP 等增强能力。我理解它们的分工是：

| 能力           | 主要价值                 |
|--------------|----------------------|
| OpenSpec     | 把需求变更规格化，辅助任务验收      |
| Beads        | 让 Phase 依赖和任务状态可追踪   |
| GitNexus     | 做调用链和影响面分析           |
| autoresearch | 补充场景、调试、归档预测         |
| 飞书 MCP       | 打通 MRD、PRD、技术方案、归档报告 |

这说明 dev-workflow 并不是单纯 prompt 工程，而是在尝试把 AI Coding 接入现有工程工具链。

## 2. 我理解的核心设计：用 Harness 约束不确定性

dev-workflow 最值得借鉴的地方，是它没有假设 LLM 会天然自律，而是通过 Harness 把不确定性包起来。

### 2.1 上下文隔离

每个阶段尽量由独立 Agent 执行，主流程只传路径和元数据，不直接塞入大段文档或代码。

这个设计有两个明显收益：

- 避免主会话 token 爆炸。
- 降低后续阶段被无关上下文污染的概率。

这也引出了一个实践问题：知识库不是越大越好，而是越可路由越好。

### 2.2 知识库分层注入

当前设计里，知识库按 L0 / L1 / L2 分层：

| 层级 | 作用                                      |
|----|-----------------------------------------|
| L0 | Agent 启动必读摘要，例如 <code>CONTEXT.md</code> |
| L1 | 按职责读取的领域、架构、核心流程文档                      |
| L2 | 深层资料，默认禁止一次性大量读取                        |

我觉得这比“全量 RAG”更克制。不同 Agent 需要的知识不同：

- PRD Agent 更需要业务与领域知识。
- 技术方案 Agent 更需要架构与设计层。
- 实现和测试 Agent 更需要核心流程与逻辑层。
- Review 和 Runner 反而应该少读背景，避免被解释性内容带偏。

### 2.3 阶段确认门

MRD 澄清、PRD、技术方案、test\_spec、代码生成等关键节点都有确认门。

确认门不是为了让人机械审批 AI，而是为了控制错误传播。需求理解越早偏，后续返工越大。确认门让人类在关键业务判断上重新接管。

### 2.4 TDD 验证闭环

`03-code-gen-tdd` 将代码阶段拆成：

代码块

- 1 Phase 0 环境和测试依赖预检
- 2 Phase 1 测试规格 / OpenSpec
- 3 Phase 2 生产代码实现
- 4 Phase 3 Code Review
- 5 Phase 4 测试代码生成
- 6 Phase 5 测试执行和覆盖率诊断
- 7 Phase 6 汇总和可选提交

这里有两个我觉得很值得讨论的顺序设计：

- test\_spec 在实现代码之前生成，避免“先实现再倒推测试”。
- 测试代码在 Code Review 之后生成，避免生产代码明显有问题时过早铺测试。

这是一种很强的流程约束，也会带来额外成本。它是否适合所有需求，需要按需求大小和风险分级。

## 2.5 失败分类和自动修复边界

Phase 5 明确规定：如果失败来自环境、外部配置、非本需求存量故障、基础设施问题，禁止自动进入“修实现 / 补测试”闭环。

这是非常关键的边界。AI 自动修复最大的问题之一，是容易把不属于本次变更的问题也吞进去，导致改动范围失控。

## 3. 当前实践中的真实体会

### 3.1 难点从“生成”变成“治理”

用 dev-workflow 后，我更明显地感受到：AI Coding 的难点正在从“让 AI 写代码”转向“治理 AI 的交付过程”。

需要治理的东西包括：

- 需求事实从哪里来。
- 技术方案是否经过确认。
- test\_spec 是否被实现污染。
- Review 是否只看本次 diff。
- 测试失败是否属于本次变更。
- 覆盖率是否使用一致口径。
- 归档后知识库是否真的更新。

这些问题不是换一个更强模型就能完全解决的，它们需要流程、状态和门禁。

### 3.2 Harness 会显著增加前期成本

dev-workflow 的投入不是零成本的。它需要：

- 初始化项目配置。
- 构建知识库。
- 整理规则和规范。
- 明确测试策略。
- 接入飞书、Beads、GitNexus 等工具。
- 维护 execution-state 和产物目录。

所以我认为它更适合中长期项目、存量系统、多人协作项目、复杂业务项目。对于极小的临时脚本或一次性改动，完整流程可能过重。

### 3.3 小需求需要轻量入口

参考实践里提到，小需求不建议每次都从 MRD 重新跑全链路。

我认同这个方向，可以按三类处理：

| 小需求类型                | 推荐方式                               |
|----------------------|------------------------------------|
| 字段名、校验条件、文案、简单 if 分支 | 直接走 03-code-gen-tdd 的 tech-only 模式 |
| 影响接口、字段、状态、MQ、DB     | 写 change-001.md ，生成增量方案后再进入 TDD    |
| 改变核心业务定义或跨应用联动       | 新建需求空间，重新走完整流程                     |

这可能需要在 dev-workflow 中形成正式入口，比如支持：

代码块

```
1  --tech-only
2  --incremental
3  --change req/<feature>/change-001.md
```

### 3.4 飞书集成很有价值，但权限和配置需要产品化

飞书 MCP 能把 MRD、PRD、技术方案、归档报告串起来，这对团队协作很有价值。

但实践中也会遇到：

- 文档权限不足。
- user / bot 身份差异。
- scope 配置不完整。
- 文档 URL、Wiki URL、Docx URL 混用。

- 安装和授权步骤容易散落在个人环境里。

这说明飞书集成不应该只是“能用”，还需要一套 doctor / setup 检查，帮助用户确认权限、身份和 scope 是否齐全。

## 4. 我希望和作者讨论的问题

### 问题 1：Orchestrator 的边界应该硬到什么程度？

当前设计要求 Orchestrator 只调度、不产出。这很干净，也能防止主会话越权。

但实践里会遇到一些轻量操作，比如补目录、修 state 字段、归一化元数据。如果全部绕 Agent，会让流程变重。

可否引入一个“安全微操作白名单”？

- 允许 Orchestrator 修复 `execution-state.md` 的元数据。
- 允许创建缺失目录。
- 允许做产物存在性校验和状态 reconcile。
- 仍然禁止生成业务产物、代码、测试和 Review 结论。

### 问题 2：execution-state、Beads、文件产物谁是事实源？

实践中可能出现三者不一致：

- state 写了 phase4 done，但 `test_file_list.md` 缺失。
- Beads 标记 done，但本地文件没有生成。
- 文件存在，但 state 没有推进。

是否需要一个统一的 reconcile 机制，在每次 resume 前自动比对：

代码块

- 1 读取 execution-state
- 2 检查关键产物
- 3 检查 Beads 状态
- 4 检查 git diff / commit hash
- 5 生成差异报告
- 6 自动修复低风险不一致
- 7 把高风险不一致交给用户确认

### 问题 3：知识库应该“简洁”还是“丰富”？

问题沉淀里提到“知识库不够丰富，应该简洁丰富”。我觉得这句话很准确。

知识库太简，会导致 Agent 缺少业务语义；知识库太大，又会造成 token 浪费和注意力污染。

可以讨论几个指标：

- CONTEXT.md 是否覆盖核心实体、接口、流程。
- api-index 是否覆盖主要入口。
- 核心流程文档是否覆盖高频需求链路。
- 最近失败的 Agent 是否能反向暴露知识库缺口。
- 是否需要“知识库覆盖率”或“知识库保鲜度”指标。

问题 4：项目规范应该放在哪个层级？

问题沉淀里提到“代码约束、规范应该在那个层级，项目集？”。

我觉得可以拆成三层：

| 层级   | 内容                       |
|------|--------------------------|
| 组织级  | 通用安全、日志、异常、代码风格、测试底线     |
| 项目集级 | 技术栈、框架版本、通用中间件、缓存和 MQ 规范 |
| 项目级  | 具体业务领域、模块边界、历史坑点、高风险链路   |

dev-workflow 是否可以支持规则继承？比如组织规则、项目集规则、项目规则按优先级合并，冲突时项目级覆盖项目集级。

问题 5：如何让 AI 写出“像本项目”的代码？

问题沉淀里提到“AI 写的代码应该符合自己的代码风格”。

这可能不能只靠 coding-standard，还需要：

- 从现有代码抽取风格样例。
- 在知识库中保留典型实现模式。
- 让 java-impl 优先搜索同类代码。
- 让 java-review 检查是否偏离项目惯用写法。
- 对注释、异常、日志、DTO 转换等形成项目级模板。

也许可以增加一个 `style-profile.md`，由知识库阶段自动生成项目风格画像。

问题 6：mock-first 是否应该有例外协议？

当前默认 mock-first 很适合提升速度和稳定性，也能避免测试连真实 DB/Redis。

但复杂遗留系统里，有些逻辑依赖 Spring 容器、AOP、配置绑定、动态代理或序列化链路。完全禁止集成测试，可能导致过度 mock，测试信心不足。

是否应该定义例外协议？

- 什么条件下允许 Spring context 测试。
- 什么条件下允许 Testcontainers。
- 单元测试和集成测试覆盖率是否分开统计。
- 例外是否需要在 tech-design 或 test\_spec 中显式说明。

## 问题 7：OpenSpec 触发条件是否应该从“人日”扩展为“风险”？

当前有按需求估算人日触发 OpenSpec 的思路，这很实用。

但我觉得还可以加入风险维度：

- 跨应用接口变更。
- 涉及资金、权限、状态机、幂等、补偿。
- 涉及对外 API 或消息契约。
- 涉及多人并行开发。
- 涉及高风险历史链路。

也就是说，OpenSpec 触发可以从“规模阈值”扩展为“规模 + 风险阈值”。

## 问题 8：自动修复什么时候必须停？

当前有重试次数上限，这是必要的。但停止条件还可以更语义化：

- 连续两次修复同一段代码但失败原因不变，停止。
- 测试失败从业务断言变成环境异常，停止。
- 修复开始扩大到非本需求文件，停止。
- 覆盖率提升收益很低，停止。
- test\_spec、技术方案、实现三者出现冲突，停止并请求人工判断。

## 问题 9：Harness 安装应该在用户级还是项目集级？

问题沉淀里提到“Harness 的安装应该在项目集而不是用户级”。

我倾向于：核心 harness 应该项目集级管理，用户级只保留运行时能力。

原因是：

- 规则和流程应该随项目演进，而不是散落在个人环境。
- 团队成员应该使用同一套 workflow。
- CI 或自动化环境也需要可复现地使用同一套 harness。
- 项目集级更适合做版本管理和变更审计。

但用户级安装也有优势：试用快、升级快、个人定制方便。这里可能需要插件市场和项目锁版本机制配合。

## 问题 10：什么时候该用什么模型？

问题沉淀里提到“什么时候改用什么模型”。我觉得可以把模型选择纳入 workflow 配置，而不是每次靠人判断。

一个可能的策略：

| 阶段            | 模型倾向        |
|---------------|-------------|
| 知识库初扫         | 成本优先，轻量模型   |
| MRD 澄清 / PRD  | 文档结构化强的模型   |
| 技术方案          | 推理更强的模型     |
| 核心代码实现        | 编码能力强的模型    |
| Review / 风险分析 | 推理和审查能力强的模型 |
| 归档总结          | 轻量模型        |

也可以根据风险自动升级：跨域、高风险链路、连续失败、多次 Review BLOCK 时切换更强模型。

## 5. 我建议的后续演进方向

### 5.1 增加 `/dev-workflow:doctor`

用于检查：

- 配置文件是否完整。
- Agent 路径是否可解析。
- 飞书 MCP 权限是否满足。
- execution-state 与文件产物是否一致。
- Beads issue 与 Phase 状态是否一致。
- 知识库是否过期。
- 插件和 CLI 版本是否匹配。

### 5.2 增加需求复杂度分级

建议形成三档流程：

| 档位   | 适用场景     | 流程                         |
|------|----------|----------------------------|
| nano | 极小修补     | tech-only + 测试             |
| lite | 常规需求     | 知识库 + 方案 + TDD             |
| full | 跨域或高风险需求 | OpenSpec + GitNexus + 完整归档 |

这样可以平衡严谨性和效率。

### 5.3 增加 Harness 自身回归测试

如果 dev-workflow 是一套工程系统，它自己也需要测试。

可以考虑：

- 用固定 example 需求跑 golden path。
- 用缺文件、权限失败、测试失败、覆盖率不足构造 failure path。
- 对每个 Skill 做 contract test：输入、允许读取、必须产出、状态更新。
- 对每个 Agent 做 fixture test：给固定输入，检查输出结构和关键字段。

### 5.4 增加项目风格画像

在知识库阶段自动生成：

代码块

```
1 style-profile.md
```

内容包括：

- 常见分层写法。
- 日志风格。
- 异常处理风格。
- DTO / Entity / BO 转换习惯。
- 测试命名和断言风格。
- 注释写法和边界。

这样可以让 AI 更容易写出贴近本项目的代码。

## 6. 总结

dev-workflow 给我的最大启发是：AI 研发流程的核心，不是把一个更大的 prompt 写得更聪明，而是把研发过程拆成可验证、可恢复、可讨论的工程协议。

它把需求理解、方案生成、代码实现、测试验证、知识沉淀串成了一条链路，也把很多过去靠人脑记忆的流程约束写进了 Skill、Agent、Rules 和状态文件里。

当然，这套 Harness 也会带来新的复杂度：规则如何维护、状态如何一致、门禁如何不拖慢效率、测试策略如何适配遗留系统、自动修复如何及时停手、安装应该个人级还是项目集级。

这些问题不是 dev-workflow 的缺点，反而说明它已经从“AI 写代码的小技巧”进入了“AI 如何参与真实工程体系”的讨论区。

我希望后续能和作者继续探讨：如何让 dev-workflow 在保持严谨的同时更轻量、更可恢复、更可测试，也更容易被团队规模化采用。